



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Inteligencia Artificial

Práctica 3: Planificación



Jorge de la Fuente Martínez
Pol Garcia Vernet
Roger Catalan Yarza

índice

Dominio del problema.....	2
Caso base.....	2
Extensión 1.....	3
Extensión 2.....	3
Extensión 3.....	3
Extensión 4.....	4
Operadores del problema.....	5
Juegos de prueba.....	7
Justificación de los tests.....	7

Dominio del problema

Para modelar el dominio y los problemas, es necesario entender las diferencias entre los elementos que los componen. De una manera resumida, un predicado es una declaración que define la propiedad de un objeto o una relación entre objetos para representar condiciones; una función es una variable que contiene un valor numérico no booleano, representando en cierta manera la cantidad; una acción es una regla que permite al sistema cambiar el estado del mundo; una variable es un elemento temporal para representar los objetos, localidades o variables; un objeto es una instancia, una entidad en el entorno de interés; el estado inicial es la configuración conjunta que representa el estado inicial; el estado final es la configuración conjunta a la que llega el sistema una vez se han utilizado las acciones definidas.

Así pues, para definir el dominio del problema, hemos definido los objetos. A partir de ellos, hemos definido sus propiedades cualitativas en los predicados y cuantitativas en las funciones. Por como se ha definido el enunciado del trabajo, hemos realizado las extensiones mediante iteraciones sobre el caso base, definiendo más objetos y por consecuencia definiendo las propiedades cualitativas y cuantitativas en predicados y funciones respectivamente mediante nuevas entradas, a la vez que hemos redefinido las entradas existentes según la necesidad para poder ajustarlas a la nueva definición del mundo. Al tener más objetos, predicados y funciones, también ha sido necesario añadir nuevas acciones y modificar las existentes según necesidades del problema.

Caso base

En el caso base, solo debemos considerar los objetos Ciudad y Hotel, asociando mediante predicados la ubicación del usuario, las conexiones entre ciudades mediante vuelos, qué hoteles ofrece una ciudad y una propiedad para definir cuando una ciudad se ha visitado. La única función necesaria para este caso es el recuento de ciudades visitadas.

Como acciones para este caso, hemos definido las acciones de viajar entre ciudades, donde dada una ciudad origen, llegaremos a una ciudad destino; alojarse en un hotel perteneciente a la ciudad de destino que no haya sido visitada.

Extensión 1

Para esta extensión, se han añadido tres predicados nuevos, uno para definir el punto de inicio, otro para confirmar que la ciudad dispone de hotel y otro para vincular el hotel con la visita de la ciudad. También se han añadido cinco funciones para definir los días que se pasan en una ciudad, la duración total del viaje y las condiciones de mínimo, máximo y total de días.

En cuanto a acciones, se han ajustado las acciones existentes de alojarse y viajar para considerar las características del nuevo escenario, y se ha añadido la nueva acción para pasar un día en una ciudad, que modifica los contadores para poder cumplir con las condiciones.

Extensión 2

La segunda extensión consiste en una pequeña modificación sobre la extensión 1, añadiendo el interés de las ciudades a visitar. En la implementación del dominio, no ha requerido de cambios en los predicados. Sin embargo, para las funciones, se ha añadido la función interés, que ha requerido a su vez modificar la acción de pasar el día, para que se considere este nuevo parámetro e incremente el valor del interés total.

Extensión 3

La tercera extensión ha requerido de cambios considerables en la implementación del dominio. Los vuelos pasan de ser un predicado a ser un objeto, y en consecuencia hemos cambiado el nombre del predicado a conexión para definir las posibles combinaciones entre un punto de origen y los destinos a los que se puede llegar. Comparado con la extensión 2, hemos cambiado el interés total por el coste total, hemos sustituido la función de interés de una ciudad por el coste del vuelo y el coste del alojamiento, reflejando las expensas a cubrir por el usuario.

Como consecuencia de estos cambios, hemos necesitado modificar las acciones de viajar y alojamiento. La acción de viajar ha sido modificada para añadir como prerequisite la conexión entre ciudades y el efecto de aumentar el coste con el precio del vuelo. Para la acción de pasar el día, el prerequisite para alojarse pasa a considerar que el usuario esté

alojado en un hotel de la ciudad, y se ha modificado el efecto para añadir el coste de la estancia. Comparado con la extensión 2, se ha eliminado el incrementador de interés.

Extensión 4

La extensión 4 consiste en combinar los cambios de las extensiones 2 y 3, en las que ambas han requerido de mejorar la extensión 1.

Para esta extensión, nos hemos basado en la extensión 3 para incluir los cambios realizados en la extensión. Sobre la extensión 3, se han añadido las dos funciones para incluir el interés total y el interés que ofrece una ciudad.

Para las acciones, solo la acción de pasar el día se ha modificado, añadiendo a los efectos el incremento del interés total por valor del interés que ofrece una ciudad.

Con estos cambios, tenemos definido completamente el dominio con las características requeridas en el enunciado.

Operadores del problema

La definición de los operadores del problema es bastante directa. Explicando el caso más complejo, referido al caso de la extensión 4, hemos planteado el problema de la forma más completa.

Primero, hemos definido las diferentes instancias de objetos. Es decir, hemos creado instancias de las diferentes ciudades que proponemos en el problema, hoteles y vuelos (estos en las extensiones 3 y 4).

Posteriormente, se inicializan los valores utilizando las funciones definidas en el dominio, creando el estado inicial, inicializando los contadores (días pasados en cada ciudad, coste y días totales usados) a cero y el presupuesto mínimo y máximo a valores arbitrarios.

Acto seguido, hemos definido las relaciones entre instancias: el interés que despierta cada ciudad, los hoteles disponibles en cada ciudad, el coste de cada uno de estos hoteles, las conexiones de vuelo disponibles y por ende su correspondiente coste asociado.

Por último, una vez inicializados los datos, hemos especificado las condiciones para alcanzar el objetivo. En la extensión 4, las condiciones definidas han sido el número de ciudades visitadas en la que el resultado debe ser mayor o igual al valor especificado, el tiempo total que debe ser igual o superior al valor especificado; y el coste, que debe comprenderse entre los valores mínimo y máximo que hemos definido, con ambos límites incluidos. Así mismo, especificamos que el objetivo debe buscar minimizar el coste, a la vez que maximiza el interés generado por el plan de viaje mediante ponderación.

La definición del problema para las anteriores extensiones y el caso base son simplificaciones de esta propuesta de problema.

El caso base no considera ni interés, ni conexiones entre vuelos, ni tiempo invertido en el viaje ni costes, y por lo tanto las inicializaciones relacionadas con estos parámetros no se definen. El objetivo también se simplifica a considerar solo cumplir con la condición de visitar un número mínimo de ciudades, y tampoco existe una ponderación en la condición del objetivo.

Para la extensión 1 del caso base, no se considera ni el interés ni las conexiones ni costes, pero sí se definen las restricciones de duración máxima y mínima del viaje, por lo que los parámetros relacionados con estas restricciones se inicializan y el objetivo pasa a tener en consideración este aspecto.

Las extensiones 2 y 3 expanden la extensión 1, por lo que a parte de las restricciones relacionadas con la duración, para la extensión 2 se inicializan los valores del interés y se condiciona el objetivo a minimizar el interés (por condiciones del enunciado se define por orden de prioridad el interés. La ciudad más interesante tendrá valor 1, la segunda 2...). Para la extensión 3, en cambio, se inicializan los costes de los hoteles, las conexiones entre vuelos y se condiciona el objetivo a cumplir el presupuesto, buscando minimizar el coste total.

Juegos de prueba

El código de este proyecto está en una carpeta llamada *src*. Dentro de la misma hay varias carpetas, nombradas según la extensión codificada: *basic*, *ext1*, *ext2*, *ext3*, *ext4*. Cada uno de estos directorios contiene dos archivos:

- *domain.pddl*: codifica los operadores del problema.
- *problem.pddl*: base de hechos del problema. Es un programa sencillo para generar una solución válida.

También puede contener otros archivos *problemX.pddl*, siendo *X* un número: *problem1.pddl*, *problem2.pddl*, que son archivos de test. Por tanto, si se quiere ejecutar un problema de una extensión en concreto, dentro del directorio de la extensión en específico, se debe indicar al planificador que se debe analizar el archivo *domain.pddl* para los operadores, y siendo el archivo de los hechos a elección del usuario.

Justificación de los tests

Se definen los siguientes juegos de pruebas para comprobar la lógica del código PDDL:

- Nivel básico (directorio *basic*):
 - *problem.pddl*: archivo que devuelve una solución válida como ejemplo. Aquí se ve la importancia del operador (*visitada ?c*). Si no estuviese agregado, puede ocurrir el caso en que se pudiera pillar dos vuelos consecutivos sin un alojamiento de por medio.
 - *problem1.pddl*: el objetivo es encontrar un plan que minimice el número de ciudades, pudiendo ser cero el mínimo. Por tanto, la solución encuentra un plan que lo cumple, de tal forma que no hay ni vuelos ni alojamientos.
 - *problem2.pddl*: el objetivo es buscar un plan que pase, como mínimo por tantas ciudades como ciudades tiene el problema (en este caso, 5). La solución encuentra un plan con la sucesión de vuelos y alojamientos, pasando por todas las ciudades y sin repeticiones.
 - *problem3.pddl*: trata de buscar un plan que pase por una cantidad de ciudades igual a las registradas en el problema más uno. Evidentemente, no ofrece solución.

- *problem4.pddl*: trata de buscar un plan que pase, como máximo, por tantas ciudades como las que hay registradas en el problema. El problema es que el planificador trata de minimizar la solución. Por tanto, devuelve un plan válido en el que se producen cero vuelos y cero alojamientos, siendo una solución correcta.
- Extensión 1 (directorio *ext1*):
 - *problem.pddl*: archivo que devuelve una solución válida como ejemplo. Por cada vuelo, hay un alojamiento y, como mínimo, pasa la acción PASAR-DIA una vez, para que una ciudad se considere visitada.
 - *problem1.pddl*: se establece que el mínimo número de días en una ciudad sea mayor a su máximo número de días, causando que el problema sea irresoluble.
 - *problem2.pddl*: se establece que el mínimo número de días en una ciudad sea mayor que el mínimo número de días que debe durar el viaje completo. Con tener una ruta con un sólo destino sería suficiente. En este caso, pasa por varias ciudades, respetando el mínimo y máximo de días de estancia por cada ciudad.
 - *problem3.pddl*: se establece que el mínimo número de días que debe tener un viaje sea el producto del número de ciudades por la cantidad máxima de días que se puede estar en una ciudad. El resultado es un plan que, por cada una de las ciudades registradas, se viaja, se aloja y se está el máximo número de días posible.
 - *problem4.pddl*: igual que el test anterior, pero incrementando en uno el número mínimo de días total. Como era esperable, no existe una solución válida.
- Extensión 2 (directorio *ext2*):
 - *problem.pddl*: archivo que devuelve una solución válida como ejemplo. Existen 3 ciudades. Y a cada una de estas tiene un valor de interés diferente. La solución elige visitar las ciudades en orden respecto al interés turístico que tienen.
 - *problem1.pddl*: a todas las ciudades se les ha dado el interés máximo (valor 1), alterando el orden de la propuesta de planificación del problema anterior, tal como estaba pensado.
 - *problem2.pddl*: cada una de las ciudades están conectadas entre sí mediante un vuelo, pero ahora no existen vuelos entre la ciudad fantasma *Origen* con ninguna ciudad registrada, provocando que no se genere una solución válida.

- *problem3.pddl*: a diferencia del caso anterior, la ciudad fantasma *Origen* tiene conexión aérea con el resto de las ciudades, pero una de las ciudades no tiene conexión de inicio con ninguna otra, pero sí se puede llegar desde otra ciudad. El problema es que no existe vuelo de vuelta. Aquí se ve como dicha ciudad es la última nombrada en la planificación.
- *problem4.pddl*: los valores de interés han sido cambiados, de tal manera que todos tengan un valor diferente. También se ha cambiado que el número mínimo de ciudades a recorrer en un viaje sea uno. La solución prioriza la ciudad con menor interés en primer lugar.
- Extensión 3 (directorio *ext3*):
 - *problem.pddl*: archivo que devuelve una solución válida como ejemplo. A partir de esta extensión, existen hoteles y vuelos con diferentes precios. Dado un presupuesto mínimo y uno máximo, devuelve una planificación.
 - *problem1.pddl*: en este problema el presupuesto mínimo es mayor al presupuesto máximo. El valor del presupuesto máximo es el mismo que en el apartado anterior, del que se ha encontrado una solución. Debido a que un mínimo no puede ser mayor a un máximo, este problema no tiene solución.
 - *problem2.pddl*: el número mínimo de días de viaje se ha aumentado hasta diez, de tal forma que el presupuesto especificado en *problem.pddl* no es suficiente, lo que termina por no encontrar una solución válida.
 - *problem3.pddl*: el mínimo de días por cada ciudad de de uno, y se debe pasar por un mínimo de tres ciudades. Además, el presupuesto mínimo y máximo está delimitado a 365, que es la cantidad mínima necesaria para encontrar un trayecto válido. Aunque podría no encontrar solución debido al heurístico empleado por el planificador (que no garantiza una solución óptima), el test funciona correctamente.
 - *problem4.pddl*: en este caso, el presupuesto mínimo y máximo es igual a la suma de un vuelo y un día en la estancia del hotel más barato. El número mínimo de días en una ciudad, el número mínimo de días total y el número mínimo de ciudades a recorrer es 1. Este test muestra la importancia del (*not (visitada ?c)*) dentro del *forall* del *goal*: si no existiera, siempre se trata de hacer un recorrido con todas las ciudades registradas, a pesar de que los requisitos de este test son mucho más laxos.
- Extensión 4 (directorio *ext4*):
 - *problem.pddl*: archivo que devuelve una solución válida como ejemplo. A partir de esta extensión, existen hoteles y vuelos con diferentes precios. Existen 3 ciudades, cada una con un valor de interés distinto. Dado un

presupuesto mínimo y uno máximo, devuelve una planificación. La solución elige visitar las ciudades en orden respecto al interés turístico que tienen. Las ponderaciones del interés y el precio son de 10 y 1, respectivamente.

- *problem1.pddl*: el valor de las ponderaciones de interés y precio se modifican, resultando en 1 y 20 respectivamente. La solución prioriza el precio muchísimo más que el interés, resultando en un plan con un alto coste económico.
- *problem2.pddl*: el valor de las ponderaciones del interés y el precio son de 1 y 20, respectivamente. A todas las ciudades se les ha dado el interés máximo (valor 1), pero manteniendo la prioridad de reducir el precio por encima del interés. No se espera una variación significativa en el resultado obtenido.
- *problem3.pddl*: en el cálculo de la métrica, el factor del coste se multiplica por cero, provocando que el único factor que se considere es minimizar indicador de interés de los lugares del viaje (equivalente a *ext2*).
- *problem4.pddl*: en este caso, las diferentes ciudades tienen un valor de interés diferente, al igual que en *problem.pddl*. La diferencia reside que, en el cálculo de la métrica, el factor del coste se multiplica por cero, provocando que el único factor que se considere es minimizar el precio del viaje (equivalente a *ext3*).